Tugas Besar IF4031

Arsitektur Aplikasi Terdistribusi

Sistem Koordinasi Kebencanaan

Laporan Milestone 1

Kelompok **nasgor**

Muhammad Aulia Azka	13523137
Arlow Emmanuel Hergara	13523161
Fachriza Ahmad Setiyono	13523162

Program Studi Teknik Informatika
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung

9 Oktober 2026

Kontribusi Anggota

NIM	Nama	Kontribusi
13523137	Muhammad Aulia Azka	*Diisi anggota: perancangan, implementasi, dan penulisan laporan.*
13523161	Arlow Emmanuel Hergara	*Diisi anggota: perancangan, implementasi, dan penulisan laporan.*
13523162	Fachriza Ahmad Setiyono	*Diisi anggota: perancangan, implementasi, dan penulisan laporan.*

Kontribusi diisi manual oleh anggota agar pembagian perancangan, implementasi, dan penulisan

mencerminkan pekerjaan yang benar-benar dilakukan.

Daftar Isi

Kontribusi Anggota **i**

1 Gambaran Sistem **1**
1.1 Tujuan dan Ruang Lingkup Milestone 1 1
1.2 Arsitektur dan Batas Layanan 1
1.3 Teknologi dan Alasan Pemilihan 3
1.4 Asumsi dan Konfigurasi 4
1.5 Ringkasan Menjalankan Sistem 5
1.6 Strategi Verifikasi 5

2 P1: Interoperabilitas Data **6**
2.1 Rancangan Solusi 6
2.2 Justifikasi dan Alternatif 7
2.3 Batasan Solusi 8
2.4 Proof-of-Concept dan Keterlacakan 8

3 P2: Konkurensi dan Ketersediaan **10**
3.1 Rancangan Solusi 10
3.2 Justifikasi dan Alternatif 11
3.3 Batasan Solusi 11
3.4 Proof-of-Concept dan Keterlacakan 12

4 P3: Autentikasi dan Trust **14**
4.1 Rancangan Solusi 14
4.2 Justifikasi dan Alternatif 16
4.3 Batasan Solusi 16
4.4 Proof-of-Concept dan Keterlacakan 17

5 P4: Layanan dan Penyimpanan **18**
5.1 Rancangan Solusi 18
5.2 Justifikasi dan Alternatif 19
5.3 Batasan Solusi 20
5.4 Proof-of-Concept dan Keterlacakan 20

6 P5: Distribusi Event **22**
6.1 Rancangan Solusi 22
6.2 Justifikasi dan Alternatif 23
6.3 Batasan Solusi 24
6.4 Proof-of-Concept dan Keterlacakan 24

A Bukti dan Reproduksi Pengujian **26**

A.1 Inventaris Bukti 26
A.2 Hasil Mock yang Direkam 27
A.3 Hasil Auth dan Aggregator 27
A.4 Perintah Reproduksi 28
A.5 Bukti yang Masih Harus Dilengkapi 28

Daftar Pustaka **30**

Bab 1

Gambaran Sistem

1.1 Tujuan dan Ruang Lingkup Milestone 1

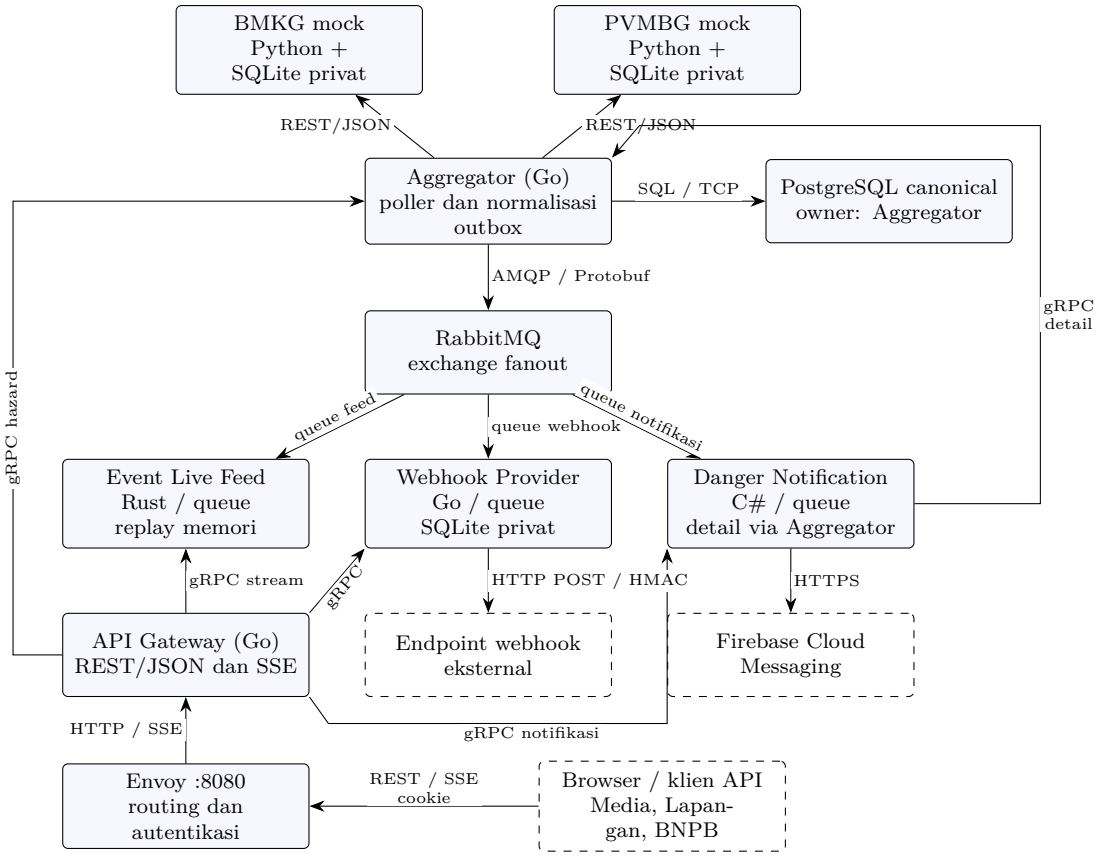
Sistem Koordinasi Kebencanaan mengintegrasikan kejadian gempa dan aktivitas gunung api dari mock BMKG dan PVMBG menjadi `HazardEvent` kanonik. BNPB mengambil data dengan polling, menyimpan hasil normalisasi, menyediakan API bagi klien, dan mendistribusikan publikasi yang sama kepada layanan live feed, webhook, serta notifikasi bahaya. Pengguna terdiri atas Media, Tim Lapangan, dan BNPB Pusat; hak akses ditentukan server berdasarkan identitas masing-masing.

Milestone ini berfokus pada interoperabilitas data (P1), isolasi kelambatan dan kegagalan sumber (P2), autentikasi dan otorisasi (P3), pemisahan layanan beserta penyimpanannya (P4), dan distribusi event melalui pub/sub (P5), sesuai spesifikasi tugas [1]. Data bencana adalah simulasi untuk pengujian, bukan informasi operasional kebencanaan. Klien membaca data hazard; operasi tulis terbatas pada pengaturan langganan, sesi, dan kontrol mock untuk demo.

Laporan merujuk kondisi kode pada 9 Oktober 2026, commit `64c03cf`. Penjelasan implementasi ditelusuri ke kode dan Compose terbaru. Beberapa uraian lama pada README masih menyebut layanan BNPB sebagai kerangka; keadaan tersebut tidak digunakan untuk menggambarkan implementasi saat ini. Keberadaan implementasi dan skrip tes dibedakan dari hasil eksekusi yang telah direkam. Tanggal serta batas cakupan bukti tersedia pada Lampiran A.

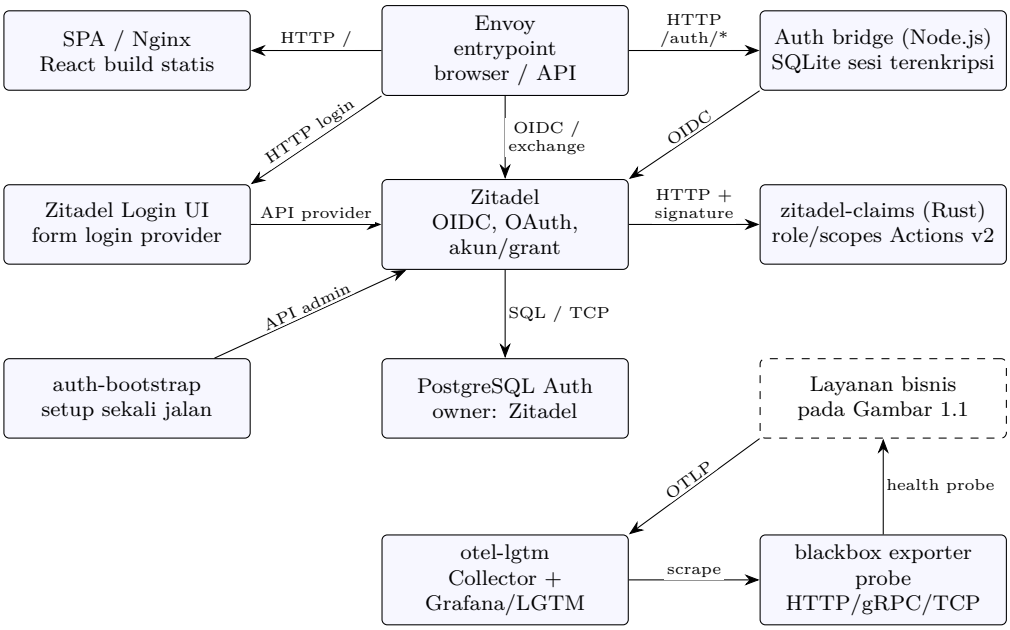
1.2 Arsitektur dan Batas Layanan

Gambar 1.1 menunjukkan jalur data, sedangkan Gambar 1.2 melengkapi komponen identitas dan observabilitas. Setiap kotak bergaris utuh merupakan container terpisah, kecuali browser dan tujuan eksternal yang ditandai khusus. SQLite dan buffer memori yang ditulis di dalam kotak dimiliki layanan tersebut, bukan database bersama. Kotak PostgreSQL adalah container database terpisah dengan pemilik logis tunggal.



Gambar 1.1: Container dan aliran data. Panah REST/gRPC mengikuti pemanggil ke penerima; panah AMQP mengikuti aliran publikasi.

1. Aggregator melakukan polling REST/JSON ke kedua mock dengan kredensial berbeda. Masing-masing mock membangkitkan data sendiri dan menyimpannya dalam SQLite privat. Tidak ada push langsung dari mock ke BNPB.
2. Aggregator memetakan data menjadi format kanonik. Hazard dan outbox dimasukkan melalui transaksi PostgreSQL; publisher mengirim envelope Protobuf dari outbox ke RabbitMQ.
3. RabbitMQ membagikan publikasi ke queue tiap consumer. Feed menyediakan replay dan stream, Webhook Provider mengirim HTTP callback, dan Danger Notification mengirim push melalui Firebase Cloud Messaging (FCM).
4. Browser mengakses satu origin melalui Envoy. API Gateway Go menerjemahkan REST/JSON dan SSE menjadi pemanggilan gRPC ke layanan bisnis. Pembacaan hazard mengambil canonical store melalui Aggregator, bukan memanggil mock secara sinkron.



Gambar 1.2: Container identitas, penyajian frontend, bootstrap, dan observabilitas. Komunikasi ke Zitadel mencakup OIDC, introspeksi, serta token exchange.

Kelompok komponen *Auth* pada rancangan awal direalisasikan oleh Zitadel, hosted Login UI, Auth bridge, dan target Actions `zitadel-claims`. Auth bridge hanya menangani `/auth/*`; permintaan API bisnis melewati Envoy dan API Gateway. Bootstrap adalah container sekali jalan untuk menyiapkan proyek, akun, grant, dan client OIDC. Konfigurasi observabilitas menggunakan container `otel-lgtm` serta `blackbox`, bukan container Prometheus dan Jaeger terpisah seperti sketsa awal. PostgreSQL Auth berbeda dari PostgreSQL canonical store. Kotak putus-putus layanan bisnis pada diagram kedua merujuk kelompok container pada diagram pertama, bukan container tambahan.

1.3 Teknologi dan Alasan Pemilihan

Komponen	**Teknologi**	**Alasan dan tanggung jawab**
BMKG dan PVMBG	Python 3.12, HTTP standard library, SQLite	Dependensi ringan; thread request memungkinkan health/admin tetap merespons saat request data menunggu. File database mempertahankan seed dan event setelah restart.
Frontend	React, TypeScript, Vite, Nginx	SPA dengan tampilan login, filter/daftar/detail hazard, status sumber, live feed, notifikasi, dan webhook; Nginx menyajikan hasil build statis.
Edge dan REST API	Envoy; Go, chi, oapi-codegen	Edge melakukan routing dan pertukaran token. Go menjaga bentuk JSON, status HTTP, validasi OpenAPI, dan framing SSE yang berbeda dari ProtoJSON.
Aggregator	Go, gRPC, PostgreSQL 17	Goroutine untuk polling independen; transaksi dan indeks relational untuk data inti, JSONB untuk atribut dinamis.

Komponen	Teknologi	Alasan dan tanggung jawab
Event Live Feed	Rust, Tokio, Tonic	Stream asinkron dan buffer replay berbatas; API Gateway mengubahnya menjadi SSE untuk browser.
Webhook Provider	Go, SQLite WAL	Menyimpan subscription, hasil delivery, dan deduplikasi; worker mengirim HTTP callback bertanda tangan.
Danger Notification	C#, ASP.NET gRPC, Firebase Admin	Memfilter bahaya SIAGA/AWAS, mengelola topik perangkat, serta mengirim push FCM. Detail dibaca dari Aggregator.
Identitas	Zitadel, PostgreSQL, Node.js 24, Rust	Provider mengelola akun; bridge menjalankan OIDC dan sesi SQLite terenkripsi; Rust Actions memasok role/scope.
Distribusi event	RabbitMQ 4, AMQP, Protobuf	Fanout dan queue independen memisahkan produsen dari consumer serta menyediakan acknowledgement.
Observabilitas	OpenTelemetry, otel-lgtm, blackbox exporter	Pengiriman trace/metric/log dan probe HTTP/gRPC/TCP; dashboard serta alert diprovisikan melalui berkas repo.
Orkestrasi	Docker Compose	Batas proses, jaringan, volume, dan dependensi dinyatakan di satu stack utama; Compose khusus dipakai untuk pengembangan parsial.

1.4 Asumsi dan Konfigurasi

1. Instalasi M1 berjalan pada satu host Docker. Port publik default adalah BNPB 8080, BMKG 8081, PVMBG 8082, Grafana 3000, serta RabbitMQ Management 15672. Port mock dalam container adalah 8080.
2. Host `localhost` dan `auth.localhost` menunjuk mesin demo. HTTP diizinkan secara eksplisit untuk lingkungan lokal; ini bukan konfigurasi TLS produksi.
3. Setiap mock menyiapkan minimal 20 record pada database baru. Interval event default 10 detik; polling contoh 3 detik. Semua nilai latensi konfigurasi merupakan parameter simulasi, bukan hasil benchmark.
4. ID sumber dan timestamp menjadi dasar watermark polling inklusif. Tabel referensi gunung statis merupakan data milik BNPB. Penerimaan field tambahan tidak menjamin kompatibilitas perubahan tipe atau penggantian nama field wajib.
5. Canonical store hanya diakses langsung oleh Aggregator untuk operasi bisnis. Layanan lain memakai gRPC atau event. Volume berisi data privat masing-masing; secret bukan bagian source code frontend.
6. Akun pengguna di-seed pada Zitadel, bukan SQLite mock ataupun frontend. Tiga akun manusia memiliki role terpisah; service user untuk tes end-to-end memiliki kredensial tersendiri.
7. Notifikasi push memerlukan proyek Firebase dan service-account JSON lokal; konfigurasi web publik diperlukan untuk push browser. Tanpa dependensi tersebut, demo push belum dapat dianggap terbukti.
8. Restart mempertahankan volume bernama. Menghapus volume menghilangkan data persisten. Replay feed berada di memori, dan RabbitMQ belum memiliki mount data persisten eksplisit pada Compose saat ini.

1.5 Ringkasan Menjalankan Sistem

Prasyarat dasar adalah Docker Desktop/Engine yang aktif dan Python 3.12+ untuk helper setup. Siapkan konfigurasi lokal dari contoh; helper mempertahankan secret yang sudah tersedia. Helper mock hanya mengisi dua kredensial sumber, sehingga nilai `REPLACE_*` lainnya dalam `.env` harus diprovisikan sesuai pemiliknya. Isi kredensial broker dan token internal yang digunakan; sesuaikan DSN database dengan definisi Compose. Sediakan file Firebase pada `FIREBASE_CREDENTIALS_PATH`. Nilai privat tidak dicantumkan dalam laporan.

```
python tools/setup_mock_env.py
python tools/setup_auth_env.py
docker compose --env-file .env --env-file .env.auth -f compose.yaml config --quiet
docker compose --env-file .env --env-file .env.auth -f compose.yaml up --build -d --wait
```

Perintah terakhir merupakan satu perintah orkestrasi setelah prasyarat konfigurasi dipenuhi. Buka `http://localhost:8080`, pilih masuk melalui Zitadel, lalu gunakan akun `media-demo`, `lapangan-demo`, atau `bnpb-demo`. Password berasal dari `.env.auth`. Dashboard Compose utama menggunakan `VITE_DATA_MODE=gateway`; fixture lokal hanya untuk mode pratinjau/stack Auth terpisah. Login UI milik Zitadel tidak mengubah frontend BNPB menjadi SSR.

Panduan operasional tersedia di `README.md`, `docs/mock-testing.md`, `docs/auth-testing.md`, `docs/e2e-testing.md`, dan `docs/load-testing.md`. Untuk layanan BNPB yang sudah berkembang, gunakan Compose dan README layanan terbaru bila uraian status pada README root masih tertinggal. Stack parsial `compose.mocks.yaml`, `compose.auth.yaml`, serta `compose.frontend.yaml` tetap berguna untuk pengembangan, tetapi bukan pengganti pengujian integrasi stack utama.

1.6 Strategi Verifikasi

Bukti dipisahkan menjadi: (1) inspeksi kode dan konfigurasi, (2) hasil tes tersimpan yang memiliki tanggal dan cakupan tertentu, serta (3) skenario yang masih perlu dieksekusi. Hasil historis mock dan browser Auth dilampirkan tanpa token maupun password. Keberadaan skrip k6 tidak diperlakukan sebagai bukti bahwa batas p95, throughput, dan error sudah tercapai. Demikian pula, keberadaan tiga consumer belum membuktikan keberhasilan skenario penghentian consumer pada stack penuh. Setiap bab menyatakan batas kesimpulan tersebut.

Bab 2

P1: Arsitektur dan Interoperabilitas Data

2.1 Rancangan Solusi

2.1.1 Kontrak Sumber dan Akuisisi Data

BMKG menyediakan `GET/seismic-events` dan `GET/tsunami-warnings`; PVMBG menyediakan `GET/volcanic-reports`. Ketiganya menerima `since` ISO 8601 yang inklusif dan mengembalikan koleksi `items`. Aggregator tidak meminta sumber mengikuti satu skema baru. Adapter per sumber memahami format asli, mengambil data dengan HTTP, lalu mengubahnya menjadi `HazardEvent`. Kontrak sumber didokumentasikan dalam `contracts/openapi/bmkg.yaml` dan `contracts/openapi/pvmbg.yaml`; kontrak keluaran publik ada pada `contracts/openapi/gateway.yaml`.

Kedua mock menggunakan SQLite file dengan kolom ID, waktu terindeks, dan payload JSON. Inisialisasi membuat tabel serta minimal 20 record ketika penyimpanan belum berisi data; seed tidak ditambahkan ulang pada setiap restart. Generator menambah satu event per interval, default 10 detik. BMKG memakai pola variasi berdasarkan indeks dan UUID untuk ID baru; tidak seluruh nilai gempa dibangkitkan acak. PVMBG memilih variasi laporan dengan bilangan acak, termasuk `confidence_level` ketika skema v2 aktif. Delay BMKG tetap, default 100 ms; delay PVMBG diambil acak antara 500–3000 ms. Sumber implementasi adalah `app.py`, `config.py`, dan `storage.py` pada direktori masing-masing mock.

2.1.2 Model Kanonik dan Pemetaan

`HazardEvent` memisahkan atribut lintas sumber dari rincian mentah. Tujuh field ringkasan adalah `hazard_id`, `source`, `hazard_type`, `severity`, `area_name`, `occurred_at`, dan `ingested_at`. Tampilan penuh menambahkan `source_ref_id`, `latitude`, `longitude`, serta `attributes`. BNPB memberikan ID kanonik dan waktu ingest saat menyimpan record. Unique key pasangan sumber dan ID asli menjaga identitas kanonik pada pengambilan berulang.

Field kanonik	BMKG	PVMBG
`source`	BMKG	PVMBG
`source_ref_id`	`event_id`	`report_id`
`hazard_type`	SEISMIC	VOLCANIC
`area_name`	`region_name`	Nama gunung dari referensi BNPB berdasarkan `volcano_id`.
Koordinat	`epicenter_lat`, `epicenter_lon`	Koordinat referensi gunung statis.
`occurred_at`	`occurred_at`	`reported_at`
`severity`	Tingkat warning terkait; jika tidak ada, berdasarkan magnitudo.	Normal/Waspada/Siaga/Awas menjadi enum huruf kapital.
`attributes`	Magnitudo, kedalaman, potensi tsunami, daftar warning, dan tambahan sumber.	Jumlah erupsi, tinggi kolom abu, ID gunung, confidence bila ada, dan tambahan sumber.

Tabel 2.1: Pemetaan ke model kanonik pada adapter Aggregator.

`BMKGPoller` mengambil gempa serta warning melalui endpoint terpisah, lalu membentuk indeks warning menurut `related_event_id`. Warning dihubungkan ke `event_id` sebelum normalisasi. Jika terdapat lebih dari satu warning, tingkat terberat dipilih dengan urutan AWAS, SIAGA, WASPADA. Tanpa warning, magnitudo $m < 5{,}0$ menghasilkan NORMAL, $5{,}0 \leq m < 6{,}5$ WASPADA, dan $m \geq 6{,}5$ SIAGA. Rincian warning disimpan sebagai array `attributes.tsunami_warnings`, sehingga informasi korelasi tetap tersedia pada proyeksi penuh.

Pemetaan PVMBG memakai `LookupVolcano` di `services/aggregator/internal/domain/volcano.go`. Implementasi menormalkan ID ke huruf kecil dan menerima awalan `v-`. Referensi memuat nama serta koordinat; dengan demikian mock tidak harus menambahkan field lokasi ke kontrak aslinya. Implementasi pemetaan kedua sumber berada di `services/aggregator/internal/ingest/mapper.go`.

2.1.3 Perubahan Skema saat Runtime

Operator mengirim `POST/admin/schema-version` dengan `{"version":2}` pada PVMBG. Pengaturan ini memengaruhi laporan yang dibuat selanjutnya, tanpa restart. Laporan lama tetap tidak memiliki `confidence_level`; laporan baru menyimpan nilai tersebut dalam JSON SQLite. Mengembalikan versi menjadi 1 tidak menghapus field yang telah tersimpan pada laporan v2.

Decoder Aggregator memiliki field `ConfidenceLevel` opsional sekaligus map `RawFields`. JSON dibaca menjadi struktur bertipe untuk field inti dan map untuk field tambahan. Field yang sudah dikenal dikeluarkan dari map, sedangkan field tidak dikenal diteruskan ke `attributes`. Kebijakan ini berlaku pada gempa, warning tsunami, dan laporan vulkanik. Nilai confidence yang tidak ada dibedakan dari nilai numerik 0. PostgreSQL menyimpan atribut dalam JSONB tanpa menambah kolom; kontrak internal membawa atribut melalui struktur dinamis Protobuf. Pemanfaatan JSONB ini sesuai kemampuan penyimpanan dan pemrosesan JSON pada PostgreSQL [6].

2.2 Justifikasi dan Alternatif

Model kanonik menghindarkan klien dari keharusan memahami dua format, dua kredensial sumber, serta aturan korelasi tsunami. Adapter terpisah membuat perubahan pada satu sumber tidak harus merombak API publik. Alternatif meneruskan payload mentah apa adanya lebih

singkat untuk dibuat, tetapi setiap klien harus menduplikasi pemetaan dan pemisahan akses Media.

Struktur inti bertipe dengan map tambahan dipilih sebagai kompromi. Deserialisasi sangat ketat yang menolak semua field baru dapat menghentikan ingest ketika confidence mulai muncul. Sebaliknya, map bebas untuk seluruh record memindahkan kesalahan tipe dan field wajib ke tahap yang lebih jauh. Implementasi saat ini mempertahankan tipe inti dan menggunakan map untuk perubahan aditif. Field tambahan dapat ditelusuri kembali dalam proyeksi penuh, tanpa memperluas tujuh field ringkasan Media.

Polling mengikuti spesifikasi dan memudahkan pengujian sumber yang tidak menyediakan push. Watermark mengurangi pengambilan data berulang, sementara upsert melindungi identitas record. Konsekuensinya, data yang terlihat klien tertinggal setidaknya selama waktu tunggu polling dan pemrosesan; sistem ini bukan integrasi real-time dengan jaminan keterlambatan nol.

2.3 Batasan Solusi

- Dukungan perubahan aditif tidak otomatis menangani rename, penghapusan field wajib, atau perubahan tipe, misalnya `magnitude` menjadi objek. Perubahan semantik tersebut memerlukan pembaruan adapter dan kontrak.
- Field yang tidak dikenal diteruskan, tetapi tidak otomatis dipahami secara bisnis. Penambahan nama yang bertabrakan dengan key atribut hasil normalisasi perlu kebijakan validasi yang lebih tegas.
- ID gunung yang tidak ditemukan menggunakan ID asli sebagai nama area dan koordinat 0,0. Nilai ini merupakan fallback implementasi, bukan koordinat valid yang sudah diverifikasi.
- Fetch warning BMKG dilakukan setelah fetch gempa. Error warning saat ini diabaikan, sehingga kejadian dapat dipetakan berdasarkan magnitudo tanpa pengayaan tsunami. Pembaruan warning untuk gempa yang sudah melewati watermark juga belum mempunyai alur pembaruan mandiri.
- JSONB tidak menghilangkan kebutuhan validasi domain. Data tidak lengkap dan enum tidak dikenal belum memiliki karantina ingest yang menyeluruh.

2.4 Proof-of-Concept dan Keterlacakan

Pada 9 Oktober 2026 dijalankan tes paket Aggregator dengan `gotest-json./internal/ingest./internal/domain./internal/auth`. Tujuh tes tingkat utama paket ingest lulus, mencakup fetch HTTP kedua sumber, penolakan kredensial, pemetaan magnitudo, korelasi warning, evolusi skema, dan proyeksi data. Paket domain tidak memiliki test case dan tidak dihitung sebagai tes lulus. Rekaman ada dalam bukti E3 pada Lampiran A.

Kriteria	**Bukti tersedia**	**Batas kesimpulan**
Pemetaan dua sumber	E3: tes mapping gempa/warning dan vulkanik lulus; `mapper_test.go`.	Membuktikan fungsi dengan input tes, belum seluruh alur Docker.
Skema v1/v2 dan field tambahan	E3: `TestMapPVMBGToHazard_SchemaEvolution`; E1: satu laporan v2 baru ditemukan saat kontrol runtime diubah.	E1 menguji mock; tes unit menguji mapper. Demo kontinu melalui gateway belum direkam.
Proyeksi kanonik	E3: `TestProjection_MediaVsFull` lulus.	Perlu E2E untuk mengikat proyeksi dengan role dan respons REST sesungguhnya.

Tabel 2.2: Bukti P1 yang tersedia dan batas cakupannya.

Untuk melengkapi demo runtime, jalankan stack utama pada skema v1, rekam satu respons hazard vulkanik penuh beserta ID dan waktu ingest, ubah skema menjadi v2, lalu tunggu generator dan polling. Verifikasi hazard baru memiliki `attributes.confidence_level`, record lama tetap dapat dibaca, dan ID/start time container mock serta Aggregator tidak berubah. Request admin harus membawa bearer PVMBG dari konfigurasi lokal. Kembalikan skema ke v1 setelah demo. Simpan respons yang sudah disanitasi, correlation ID, serta log terkait; skenario ini belum diberi status lulus pada laporan.

Bab 3

P2: Konkurensi dan Ketersediaan

3.1 Rancangan Solusi

Sistem memisahkan pengambilan data sumber dari pelayanan permintaan pengguna. Aggregator menjalankan poller BMKG dan PVMBG sebagai dua goroutine independen, sementara layanan gRPC membaca data kanonis di PostgreSQL. Publisher outbox berjalan pada goroutine lain. Pemisahan ini terlihat pada `services/aggregator/cmd/main.go`. Dengan demikian, keterlambatan PVMBG memengaruhi waktu pembaruan data PVMBG, bukan menambahkan waktu tunggu upstream ke setiap pembacaan BMKG. Permintaan pengguna tetap bergantung pada kapasitas gateway, autentikasi, Aggregator, dan database yang dipakai bersama.

Setiap poller melakukan pengambilan awal, kemudian mengulangi polling menurut interval masing-masing. HTTP client memiliki timeout per sumber dan menerima context untuk pembatalan saat layanan berhenti. Dalam satu poller, satu putaran diselesaikan sebelum putaran berikutnya diproses; implementasi tidak membuat goroutine baru tanpa batas pada setiap tick. BMKG mengambil seismic event lalu peringatan tsunami secara berurutan, sedangkan PVMBG mengambil laporan vulkanik. Implementasi berada pada `services/aggregator/internal/ingest/bmkg.go` dan `services/aggregator/internal/ingest/pvmbg.go`.

Tabel 3.1: Parameter konkurensi dan kesegaran data

Parameter	**Nilai acuan**	**Makna**
Interval polling kedua sumber	3.000 ms	Contoh konfigurasi di `.env.example`.
Timeout BMKG / PVMBG	1.000 / 3.500 ms	Batas setiap request HTTP upstream, bukan target latensi API.
Ambang stale	10.000 ms	Acuan usia keberhasilan pengambilan sumber.
Timeout unary gateway	2.000 ms	Default deadline panggilan backend.
Permintaan unary aktif	100	Default kapasitas semaphore gateway; stream dikecualikan.
Koneksi PostgreSQL	5–25	Minimum dan maksimum pool Aggregator.

Setiap sumber mempunyai watermark, waktu keberhasilan terakhir, waktu percobaan terakhir, dan pesan kegagalan. Setelah fetch berhasil, data dinormalisasi dan disimpan sebelum watermark dimajukan. Pada kegagalan, data sebelumnya tidak dihapus dan watermark keberhasilan dipertahankan. Pada pembacaan API, `services/aggregator/internal/domain/status.go` menghitung status berikut:

- **FRESH**: keberhasilan terakhir masih dalam ambang waktu dan tidak ada error sumber yang

tercatat.
- **STALE**: sumber gagal atau terlalu lama tidak berhasil, tetapi data kanonis sumber tersebut masih tersedia.
- **UNAVAILABLE**: tidak tersedia data tersimpan untuk digunakan ketika sumber belum berhasil atau tidak dapat diperbarui.

Respons hazard tetap membawa item yang tersedia beserta metadata `freshness`. Nilai `partial=true` diberikan apabila sumber yang relevan dengan permintaan tidak fresh. Filter sumber membatasi pemeriksaan kesegaran pada sumber tersebut. Mekanisme ini membedakan kondisi tidak ada peristiwa dari sumber yang belum dapat memberikan data, sekaligus memungkinkan klien menampilkan usia data. Pembentukan respons ada pada `services/aggregator/internal/grpc/server.go`; penyimpanan watermark ada pada `services/aggregator/internal/store/hazard_store.go`.

Gateway juga membatasi permintaan unary aktif dengan channel berkapasitas tetap. Ketika penuh, permintaan ditolak dengan respons overload sesuai kontrak, umumnya HTTP 429, dan `Retry-After`. Deadline unary membatasi waktu tunggu backend. Pembatas ini bukan pembatas laju per pengguna dan tidak menghitung stream SSE yang berumur panjang. Kode terkait adalah `services/api-gateway/internal/server/server.go`, `services/api-gateway/internal/config/config.go`, dan `services/api-gateway/internal/problem/problem.go`.

3.2 Justifikasi dan Alternatif

Polling independen dipilih karena data kebencanaan dikumpulkan secara berkala dan dapat dilayani dari penyimpanan kanonis. Pengguna memperoleh respons dari satu bentuk data meskipun kedua sumber memiliki latensi serta skema berbeda. Biaya keputusan ini adalah jeda antara perubahan upstream dan kemunculannya pada API. Karena itu, freshness merupakan bagian dari kontrak respons, bukan sekadar informasi log.

Alternatif pertama ialah melakukan dua panggilan upstream secara serial pada setiap request. Pendekatan tersebut sederhana, tetapi latensi sumber terakumulasi dan gangguan satu sumber langsung memengaruhi request pengguna. Alternatif kedua ialah fan-out paralel per request dengan deadline terpisah. Pendekatan kedua mengurangi penjumlahan waktu tunggu, tetapi tetap menambah beban upstream sebanding dengan jumlah pengguna dan memerlukan fallback tersendiri.

Penyimpanan kanonis dengan polling mengurangi keterikatan tersebut, dengan konsekuensi kapasitas database menjadi sumber daya bersama. Pool koneksi dan batas unary gateway menahan pertumbuhan pekerjaan aktif. Penggunaan circuit breaker atau penjadwal polling terdistribusi dapat menjadi pengembangan berikutnya, tetapi keduanya belum menjadi mekanisme yang digunakan pada implementasi ini.

3.3 Batasan Solusi

Isolasi pada tingkat goroutine tidak menjamin isolasi CPU, memori, database, atau autentikasi. Permintaan BMKG dan PVMBG tetap melewati komponen bersama. Kegagalan PostgreSQL tidak dapat ditutupi oleh fallback memori karena jalur baca kanonis masih membutuhkan database. Compose juga belum memberikan bukti failover beberapa replika Aggregator atau database.

Status kesegaran BMKG terutama mengikuti keberhasilan seismic event. Error pengambilan peringatan tsunami saat ini diabaikan oleh poller, sehingga kegagalan pengayaan tsunami belum selalu muncul sebagai status stale. Selain itu, pembaruan watermark dilakukan sesudah

transaksi data selesai, bukan di dalam transaksi yang sama. Restart pada sela kedua operasi dapat menyebabkan data diambil kembali; kestabilan identitas hazard membantu menjaga baris kanonis, tetapi tidak membuktikan tidak ada publication berulang.

Nilai timeout, kapasitas pool, serta semaphore merupakan konfigurasi atau mekanisme yang ditemukan pada kode. Nilai tersebut tidak membuktikan target p95, throughput, maupun tingkat availability tertentu tanpa pengukuran. Tidak ada hasil uji beban gateway terbaru yang dilampirkan untuk snapshot laporan 9 Oktober 2026.

3.4 Proof-of-Concept dan Keterlacakan

Bukti historis container pada 30 September 2026 disimpan dalam `docs/tex/evidence/mock-container-2026-09-30.json`. Catatan tersebut memuat 10 pemeriksaan berhasil pada lingkup mock. Pengukuran langsung BMKG ketika empat request PVMBG berjalan paralel tercatat 113,86 ms; pemeriksaan health dan admin PVMBG tercatat masing-masing 22,29 ms dan 17,80 ms. Angka ini menunjukkan perilaku pada skenario mock yang dicatat, bukan p95 gateway atau hasil 50 pengguna selama 60 detik.

Tabel 3.2: Status pembuktian P2 pada snapshot laporan

Skenario	**Target atau cakupan**	**Hasil**
Konkurensi mock	Request BMKG bersamaan dengan empat request PVMBG	Bukti historis tersedia.
Beban API lengkap	50 VU, 60 detik; error tidak terkendali di bawah 1%. Skrip juga menetapkan p95 di bawah 2 detik	Belum diukur.
Isolasi melalui gateway	PVMBG ditunda 3 detik; p95 grup BMKG di bawah 300 ms	Belum diukur.
Degradasi sumber	Data lama tetap terbaca dengan freshness yang sesuai	Mekanisme tersedia; hasil terbaru belum dilampirkan.

Repositori menyediakan skenario yang dapat diulang pada `tests/load/aggregator.js` dan `tests/load/aggregator-isolation.js`, dengan petunjuk pada `docs/load-testing.md`. Prosedur pembuktian yang direncanakan adalah:

1. Jalankan stack yang telah diprovisikan, pastikan akun layanan tersedia pada `.cache/auth/`, dan buat direktori `.cache/load/`. Jalankan `k6runtests/load/aggregator.js` dari root repositori. Defaultnya adalah pemanasan 10 detik, lalu 50 VU selama 60 detik. Campuran request adalah 60% daftar, 30% detail, dan 10% status sumber. Hanya fase beban dihitung; kegagalan pengambilan token dicatat terpisah.
2. Untuk isolasi, tetapkan `PVMBG_RESPONSE_DELAY_MIN_MS` dan `PVMBG_RESPONSE_DELAY_MAX_MS` menjadi 3000, lalu buat ulang `pvmbg-mock` sebagaimana panduan. Jalankan `k6runtests/load/aggregator-isolation.js`. Setup memeriksa tiga request langsung ke mock dan membatalkan pengujian bila median delay kurang dari 2900 ms.
3. Bandingkan fase baseline 30 detik dengan fase kontensi 60 detik, setelah pemanasan 5 detik. Kedua fase menggunakan 10 VU BMKG yang diukur dan 10 VU kelompok kedua; kelompok kedua beralih dari BMKG ke PVMBG. Simpan p50/p95/p99, error, throughput, ringkasan JSON, dan jendela waktu untuk pencocokan trace. Pulihkan konfigurasi delay setelah selesai.

Latensi yang diukur skrip tersebut mencakup Envoy, token exchange, gateway, RPC, dan pembacaan database. Hasilnya harus dilaporkan sebagai latensi jalur API lengkap. Keberadaan

skrip, threshold, dan instrumen observabilitas merupakan keterlacakan rancangan pengujian; belum merupakan hasil eksekusi pengujian.

Untuk menilai batas error spesifikasi, HTTP 429 yang merupakan backpressure terkendali dilaporkan terpisah dari error tidak terkendali. Pada fase beban, catat jumlah request API N, jumlah 429 N_{429}, dan kegagalan lain N_{err}; laporkan $100N_{\text{err}}/N$ serta $100N_{429}/N$. Denominator dan definisi kegagalan harus dicantumkan agar angka dapat diperiksa. Threshold error bawaan skrip masih menghitung 429, sehingga lebih ketat daripada penghitungan penerimaan yang mengecualikannya. Hasil token endpoint tidak dicampur ke persentil request API.

Tabel 3.3: Format hasil beban yang belum diisi dengan pengukuran

Metrik	**Beban 50 VU / 60 detik**	**BMKG saat PVMBG 3 detik**
Jumlah request / throughput	Belum diukur	Belum diukur
p50 / p95 / p99	Belum diukur	Belum diukur
Error tidak terkendali / 429	Belum diukur	Belum diukur

Untuk demo outage, catat respons awal kedua jenis hazard, aktifkan `/admin/outage` PVMBG, lalu ulangi request BMKG-only dan volcanic. Data vulkanik terakhir harus tetap dapat dibaca dengan `stale_since`, atau status unavailable dinyatakan bila data belum ada. Pertahankan gangguan selama durasi demo yang dipilih, rekam log dan waktu percobaan, kemudian nonaktifkan outage dan tunggu poller pulih tanpa restart. Bukti mock historis hanya mencakup 10,64 detik, sehingga tidak membuktikan skenario outage panjang maupun perilaku fallback API BNPB.

Bab 4

P3: Autentikasi, Otorisasi, dan Trust

4.1 Rancangan Solusi

4.1.1 Pemisahan Domain Kepercayaan

BMKG menerima API key melalui `X-BMKG-Key`; PVMBG menerima token opaque melalui `Authorization:Bearer`. Keduanya memvalidasi secret milik sendiri. Endpoint data dan kontrol PVMBG terlindungi, sedangkan health dapat dipakai tanpa kredensial. Aggregator mengirim header yang sesuai pada adapter sumber; kredensial pengguna BNPB tidak diteruskan ke mock.

Helper `tools/setup_mock_env.py` membuat dua nilai acak berbeda dengan generator secret, menolak penggunaan nilai sama, dan mempertahankan secret yang sudah valid. Helper hanya memprovisikan konfigurasi; layanan dapat dijalankan tanpa helper bila environment sudah diisi. SQLite menyimpan data mock, bukan menjadi sumber API key. Seorang anggota tim yang menyiapkan environment sendiri memperoleh secret lokal sendiri dan harus menyamakan konfigurasi pemanggil serta penerima pada instalasinya.

Untuk pengguna, Zitadel menjadi identity provider sekaligus authorization server. OIDC memberikan identitas login melalui ID token; OAuth menyediakan access token untuk akses API [3]. Akun `media-demo`, `lapangan-demo`, dan `bnpb-demo` mempunyai subject dan kredensial terpisah di PostgreSQL Zitadel. Bootstrap mengatur role, grant, aplikasi OIDC, serta TTL access token default 60 detik. Akun service untuk tes API juga diprovisikan terpisah. SQLite Auth bridge hanya menyimpan sesi dan transaksi login.

4.1.2 Login, Cookie, dan Pemanggilan API

Alur login menggunakan Authorization Code dengan PKCE S256. PKCE mengikat pertukaran authorization code dengan verifier transaksi, sehingga kepemilikan code saja tidak cukup untuk menukarnya [4].

1. Browser membuka `/auth/login`. Bridge membuat state, nonce, verifier PKCE, dan cookie transaksi, lalu mengarahkan pengguna ke hosted Login UI Zitadel. Password dikirim ke provider.
2. Callback memvalidasi transaksi browser dan menukar authorization code. Library `open id-client` memeriksa ID token, issuer, audience, nonce, serta masa berlaku. Introspeksi access token harus menghasilkan identitas aktif dengan audience project dan role/scope yang dikenal.
3. Bridge menyimpan refresh token serta payload sesi dalam SQLite yang dienkripsi AES-256-GCM. Browser memperoleh cookie `bnpb_access` berisi access token opaque asli dan `bnpb_s ession` berisi ID sesi acak.

4. Kedua cookie bersifat HttpOnly, SameSite=Lax, host-only, dan Path=/; Secure diaktifkan pada HTTPS. Respons `/auth/session` hanya mengembalikan profil, scope, expiry, dan CSRF token. Token akses/refresh tidak dimasukkan ke JSON ataupun penyimpanan JavaScript.
5. Untuk `/api/*`, filter Lua Envoy memindahkan access cookie menjadi header bearer dan menghapus cookie sebelum meneruskan request. Campuran bearer eksplisit dan cookie Auth ditolak agar sumber kredensial tidak ambigu.
6. Filter token exchange Envoy menggunakan confidential client ke Zitadel dan meminta JWT melalui mekanisme RFC 8693 [5]. JWT diteruskan melalui API Gateway sebagai metadata otorisasi gRPC. Service bisnis memverifikasi token dan memeriksa izin method.

Target Rust `zitadel-claims` menerima hook Actions v2 sebelum pembentukan access token dan userinfo/introspeksi. Target memeriksa signature HMAC-SHA256 beserta timestamp panggilan dari Zitadel, lalu membentuk `role` dan `scopes` dari grant pada project/organisasi yang diizinkan. Claim tidak ditentukan oleh input role di frontend. Target ini tidak dibuka sebagai endpoint publik pada Envoy.

4.1.3 Hak Akses dan Pembatasan Data

Hak akses	MEDIA	FIELD_TEAM	BNPB_INTERNAL
Hazard ringkasan	Diizinkan	Diizinkan	Diizinkan
Hazard penuh/mentah	Ditolak	Diizinkan	Diizinkan
Baca notifikasi	Diizinkan	Diizinkan	Diizinkan
Baca webhook	Diizinkan sesuai kepemilikan	Diizinkan sesuai kepemilikan	Diizinkan sesuai kepemilikan
Buat/ubah webhook	Ditolak	Diizinkan	Diizinkan

Tabel 4.1: Scope role yang dibentuk oleh target Actions. Pemeriksaan izin berada pada backend.

Untuk hazard, Aggregator menolak permintaan `VIEW_FULL` tanpa `hazards:read:full`. Proyeksi summary hanya memuat tujuh field ringkasan Bab 2; koordinat, referensi asli, dan attributes tidak disertakan. API Gateway menerjemahkan representasi Protobuf menjadi bentuk OpenAPI dan menolak bentuk backend yang melanggar proyeksi. Menyembunyikan tombol detail pada UI hanya mendukung pengalaman pengguna; pengamanan utama adalah pengecekan scope dan pemilihan proyeksi pada server.

4.1.4 Refresh, Logout, dan CSRF

SPA menjadwalkan refresh sekitar lima detik sebelum expiry serta dapat memulihkan sesi setelah 401. Bridge menggunakan refresh token server dan menggabungkan request refresh bersamaan untuk sesi yang sama. Rotasi data sesi dan pencatatan hash token yang ditolak dilakukan dalam transaksi SQLite. Pengguna tidak perlu memasukkan password lagi selama refresh token dan sesi masih berlaku. Kode bridge juga mencoba mencabut access token lama pada Zitadel setelah rotasi.

POST refresh/logout mensyaratkan Origin aplikasi dan kecocokan nilai `X-CSRF-Token` dengan sesi. Pada mutasi API berbasis cookie, Envoy memeriksa Origin dan keberadaan header CSRF; Envoy tidak melakukan perbandingan dengan SQLite bridge. Kebijakan ini bergantung pada origin yang tepat dan tidak dibukanya CORS untuk origin sembarang. Bearer eksplisit bagi klien API mengikuti jalur tersendiri tanpa cookie browser.

Logout menghapus sesi lokal, membersihkan cookie, dan mencoba revokasi provider. Browser diarahkan ke logout Zitadel untuk mengakhiri SSO. Login UI V2 dapat menampilkan tindakan *End Session*; penghapusan cookie aplikasi saja tidak selalu mengakhiri sesi provider. Implementasi ada di `services/auth-bridge/src/server.mjs`, `provider.mjs`, dan `store.mjs`.

4.2 Justifikasi dan Alternatif

API key/opaque token sederhana dipakai untuk mock agar perbedaan protokol autentikasi antarsumber tetap terlihat tanpa membangun provider untuk setiap instansi. Pemisahan secret membatasi penggunaan kredensial lintas instansi. Konsekuensinya, rotasi dan distribusi secret mock tetap menjadi tugas konfigurasi.

OIDC terpusat menghindari pembuatan form password, hash password, dan penerbitan JWT sendiri di frontend BNPB. Dibanding menyimpan bearer token dalam localStorage, cookie HttpOnly mengurangi keterbacaan token oleh JavaScript, tetapi menambah kebutuhan perlindungan CSRF. Bridge dipakai karena refresh token serta client secret perlu berada di server. Dibanding introspeksi pada setiap service, token exchange di edge memberikan format JWT untuk validasi backend. Biayanya adalah dependensi pada provider pada jalur masuk dan tambahan konfigurasi client.

Alternatif mTLS dapat mengautentikasi identitas workload antarlayanan dengan lebih kuat, tetapi memerlukan pengelolaan sertifikat. Implementasi M1 menggunakan JWT pengguna serta token internal `x-service-token` pada jalur yang mendukungnya. Token internal bukan identitas pengguna dan tidak boleh diberikan kepada browser. Pembagian broker user per consumer akan memberi isolasi lebih baik daripada satu akun broker bersama, tetapi belum diterapkan seragam pada Compose.

4.3 Batasan Solusi

- Revokasi access token lama pada provider dilakukan asinkron dan kegagalannya ditoleransi. Daftar penolakan lokal bridge tidak dibaca Envoy. Karena itu penolakan seketika pada gateway tidak dapat dijamin jika revokasi gagal; token lama mungkin berlaku hingga expiry. JWT hasil exchange yang sudah diterbitkan juga memiliki siklus validasinya sendiri.
- Validasi audience belum seragam: Aggregator memeriksanya bila dikonfigurasi, sedangkan Danger Notification memiliki `ValidateAudience=false`. Verifier Aggregator memberi toleransi waktu 30 detik; ini perlu dibedakan dari expiry pada token opaque yang dicek provider di edge.
- Compose lokal memakai HTTP dan gRPC plaintext, bukan mTLS. Koordinasi refresh dan SQLite sesi ditujukan untuk satu instance bridge; replikasi membutuhkan koordinasi/penyimpanan bersama.
- Secret mock, Auth, PAT, dan file Firebase dikeluarkan dari Git, tetapi Compose canonical database masih menulis password pengembangan literal. Selain itu beberapa service menerima seluruh `.env` dan akun RabbitMQ bersama. Dengan demikian ketentuan tanpa secret hardcoded dan distribusi secret minimum belum terpenuhi menyeluruh; perlu perbaikan konfigurasi sebelum klaim tersebut dinyatakan lulus.
- Provider yang tidak tersedia dapat menyebabkan token exchange gagal meskipun data hazard masih tersimpan. Graceful degradation terhadap mock pada P2 tidak otomatis menghilangkan dependensi Auth ini.

4.4 Proof-of-Concept dan Keterlacakan

Kriteria	Bukti tersedia	Batas kesimpulan
Isolasi kredensial BMKG/PVMBG	E1: tujuh request tidak sah ditolak dalam tes isolasi.	Hasil mock historis 30 September; bukan audit seluruh secret BNPB.
Tiga identitas, cookie, refresh	E2: 45 pemeriksaan browser/provider nyata lulus pada 6 Oktober.	Berlaku pada arsitektur API melalui bridge saat tes direkam, bukan verifikasi gateway terbaru.
Validasi JWT backend	E3: 15 tes utama Auth Aggregator lulus pada 9 Oktober.	Mencakup issuer, expiry, algoritme, rotasi JWKS, service token, dan provider unavailable dengan server tes.
Media hanya ringkasan	E3: proyeksi summary/full lulus. Tes E2E role tersedia di `tests/test_e2e_hazards.py`.	Belum dilampiri rekaman E2E tiga role pada stack penuh terbaru.
Expiry 60 detik dan token lama	E2 merekam expiry alami serta refresh tanpa password.	Perlu tes ulang melalui Envoy; keterbatasan revokasi asinkron tetap berlaku.

Tabel 4.2: Bukti P3 dipisahkan berdasarkan jalur dan waktu pengujian.

Urutan demo P3 adalah: uji request tanpa kredensial dan kredensial silang pada kedua mock; login sebagai Media, tampilkan tujuh field lalu buktikan permintaan full ditolak; login sebagai Tim Lapangan, simpan token uji secara privat, tunggu expiry alami 60 detik, lalu amati refresh otomatis; ulangi akses dengan token lama dan pastikan ditolak pada gateway. Akhiri dengan logout serta login BNPB menggunakan akun berbeda. Tangkapan layar/log hanya menampilkan waktu, role, status HTTP, dan correlation ID; nilai token, cookie, password, dan client secret harus disembunyikan. Panduan rinci tersedia di `docs/auth-testing.md` dan `docs/e2e-testing.md`. Percobaan tes bridge lokal saat penulisan laporan belum berhasil dimulai karena dependensi OpenTelemetry pada instalasi Node lokal belum tersedia; percobaan tersebut tidak dihitung sebagai hasil lulus.

Bab 5

P4: Layanan dan Penyimpanan

5.1 Rancangan Solusi

Pemisahan layanan mengikuti tanggung jawab pengolahan data dan cara penyampaiannya. Aggregator mengintegrasikan sumber eksternal ke model kanonis; Event Live Feed melayani aliran event; Webhook Provider mengirim callback; Danger Notification memilih peringatan untuk push notification. Gateway menerjemahkan kontrak HTTP menjadi panggilan gRPC, sehingga layanan domain tidak perlu mengimplementasikan ulang seluruh antarmuka publik.

Consumer telah diimplementasikan dalam tiga bahasa: Rust untuk Event Live Feed, Go untuk Webhook Provider, dan C# untuk Danger Notification. Kontrak Protobuf menjadi batas pertukaran data lintas bahasa. Compose membangun masing-masing image dari direktori layanannya dan menghubungkannya melalui jaringan Docker. Pemisahan proses memungkinkan restart satu consumer tanpa harus mematikan consumer lain, meskipun kemandirian operasional tetap dibatasi dependency yang dipakai bersama.

Tabel 5.1: Pembagian tanggung jawab dan penyimpanan layanan

Layanan	**Teknologi**	**Tanggung jawab dan data**
Aggregator	Go, gRPC 9091	Normalisasi BMKG/PVMBG; hazard kanonis, watermark, dan outbox pada PostgreSQL.
Event Live Feed	Rust, gRPC 9093	Subscription event, filter/proyeksi, cursor, dan replay terbatas dalam memori proses.
Webhook Provider	Go, gRPC 9094	Subscription webhook, delivery log, serta deduplikasi pengiriman pada SQLite.
Danger Notification	C#, gRPC 9095	Filter SIAGA/AWAS, pembacaan melalui Aggregator, dan pengelolaan topic perangkat melalui FCM.
API Gateway	Go, HTTP	Validasi kontrak, pemetaan HTTP–gRPC, dan SSE; tidak menjadi pemilik data hazard.

Aggregator memiliki tabel `hazard_events` dengan kunci `hazard_id` dan batas unik pasangan `source` serta `source_ref_id`. Atribut umum disimpan sebagai kolom bertipe, sedangkan atribut yang berbeda menurut sumber disimpan dalam JSONB. Kombinasi ini memungkinkan filter lintas sumber tanpa membuang informasi khusus sumber. Tabel `ingest_watermarks` menyimpan kemajuan ingestion dan kesegaran, `outbox_events` menyimpan publication yang belum dikirim, dan `outbox_dead_letters` menyimpan publication yang mencapai batas gagal. Schema yang diterapkan saat startup berada pada `services/aggregator/internal/store/postgres.go`.

Kepemilikan database tetap pada Aggregator. Danger Notification memperoleh daftar/detail hazard melalui adapter gRPC, bukan dengan membaca tabel Aggregator secara langsung. Adapter tersebut berada pada `services/danger-notification/BNPB.Notification.Server/Adapters/Outbound/AggregatorClient.cs`. Pemisahan ini menjaga kontrak data antarlayanan, tetapi menjadikan pembacaan notification bergantung pada availability Aggregator. FCM mengelola subscription token ke topic; implementasi notification saat ini tidak mendaftarkan database lokal untuk inbox, riwayat pengiriman, atau deduplikasi.

Webhook Provider memiliki database SQLite tersendiri untuk `webhook_subscriptions`, `webhook_deliveries`, dan `delivery_dedup`. Mode WAL digunakan dan pool dibatasi satu koneksi terbuka. Kunci gabungan `event_id` dan `webhook_id` pada tabel dedup menandai delivery yang berhasil. Implementasi berada pada `services/webhook-provider/internal/store/sqlite.go`. Compose memasang volume `webhook_data` pada `/data`; DSN contoh mengarahkan database ke `file:/data/webhook.db`.

Event Live Feed menggunakan abstraksi `ReplayStore` dengan implementasi `MemoryStore`. Buffer menyimpan urutan event dan himpunan ID yang sudah diterima, dilindungi mutex, lalu menyebarkan event ke kanal broadcast. Default Compose membatasi replay pada 600 detik atau 10.000 entri, sedangkan kanal live memiliki kapasitas 256 entri. Ketika subscriber tertinggal, backlog dapat dibaca kembali selama belum keluar dari jendela replay. Restart membentuk epoch baru dan cursor lama ditolak, sehingga klien mengetahui bahwa riwayat sebelumnya tidak lagi tersedia. Sumbernya adalah `services/event-live-feed/src/core/ports.rs` dan `services/event-live-feed/src/adapters/outbound/memory_store.rs`.

5.2 Justifikasi dan Alternatif

Pemilihan penyimpanan mengikuti kebutuhan masing-masing layanan. PostgreSQL memberikan transaksi untuk menyimpan hazard dan outbox bersama, keunikan identitas sumber, serta indeks untuk penyaringan dan paginasi. SQLite memadai untuk volume subscription dan log webhook pada satu instance, dengan biaya operasi rendah. Memori proses menyediakan replay live-feed yang ringan ketika kontraknya memang membatasi umur riwayat.

Tabel 5.2: Alternatif penyimpanan dan pertimbangannya

Pilihan	**Kelebihan untuk kasus ini**	**Biaya atau batasan**
PostgreSQL, dipakai Aggregator	Transaksi hazard–outbox, constraint identitas, dan kolom JSONB untuk atribut sumber.	Memerlukan proses database dan pengelolaan koneksi; kapasitasnya dipakai seluruh pembacaan kanonis.
SQLite, dipakai webhook	Penyimpanan lokal persisten tanpa server database tambahan; schema dan dedup mudah diperiksa.	Satu penulis dan satu volume lokal membatasi replikasi horizontal.
MongoDB, alternatif	Dokumen fleksibel untuk payload sumber yang beragam.	Perlu merancang kembali indeks gabungan, transaksi publication, dan kontrak query; fleksibilitas payload sudah dapat ditampung JSONB.
Redis, alternatif replay	Data replay dan dedup dapat ditempatkan di luar proses feed untuk dipakai beberapa instance.	Menambah dependency; TTL, eviction, persistensi, dan posisi cursor perlu dirancang eksplisit.

Alternatif satu database bersama untuk semua layanan mengurangi jumlah komponen, tetapi memungkinkan perubahan schema satu layanan memengaruhi layanan lain secara langsung. Rancangan yang dipilih membatasi akses melalui kontrak gRPC atau event. Fleksibilitas penyimpanan berarti implementasi penyimpanan dapat disesuaikan per kebutuhan; bukan berarti

mengganti database tanpa migrasi data, perubahan adapter, dan pengujian ulang.

Untuk feed, penyimpanan eksternal seperti Redis atau PostgreSQL dapat memperpanjang replay lintas restart. Implementasi saat ini memilih memori, sehingga biaya serialisasi dan I/O penyimpanan lebih kecil, tetapi tidak mengklaim riwayat permanen. Untuk webhook berskala lebih besar, PostgreSQL dapat menggantikan SQLite agar beberapa worker mempunyai state dedup bersama, dengan tambahan koordinasi pekerjaan dan pengendalian duplikasi.

5.3 Batasan Solusi

Pemisahan image dan proses belum membuktikan bahwa setiap layanan dapat direplikasi tanpa perubahan. Publisher outbox belum memakai mekanisme claim/lock antar-worker pada query pengambilan record. Feed memiliki replay dan epoch lokal per proses. SQLite webhook membatasi penulisan serta memerlukan strategi berbagi state sebelum menjalankan beberapa replica. Keterbatasan ini perlu diselesaikan sebelum menyatakan dukungan penskalaan horizontal yang transparan.

Durability harus dibedakan per komponen. PostgreSQL menggunakan volume `aggregator_data`; SQLite menggunakan `webhook_data`. Data tersebut tetap bergantung pada lifecycle volume dan prosedur backup. Feed tidak persisten setelah restart. Notification belum menyimpan bukti pengiriman dan dedup lokal. RabbitMQ mendeklarasikan queue durable, tetapi `compose.yaml` belum memasang volume data broker secara eksplisit. Karena itu, ketahanan seluruh state setelah container dibuat ulang belum menjadi jaminan sistem.

Penyimpanan hazard mempertahankan satu identitas kanonis per referensi sumber, tetapi upsert tidak membandingkan seluruh isi untuk memastikan perubahan substantif. Publication baru tetap dapat dibentuk untuk data yang diambil ulang. Pemisahan database dan penggunaan constraint tidak dengan sendirinya menyediakan pemrosesan event tepat satu kali.

5.4 Proof-of-Concept dan Keterlacakan

Bukti implementasi dapat ditelusuri dari `compose.yaml`, Dockerfile layanan, kontrak pada `contracts/proto/`, dan adapter penyimpanan yang dijelaskan di atas. Untuk tiga consumer, titik masuk proses adalah `services/event-live-feed/src/main.rs`, `services/webhook-provider/cmd/main.go`, dan `services/danger-notification/BNPB.Notification.Server/Program.cs`. Artefak tersebut menunjukkan bahwa batas layanan dan pilihan penyimpanan telah diwujudkan dalam kode pada snapshot 9 Oktober 2026.

Bukti container historis pada `docs/tex/evidence/mock-container-2026-09-30.json` mencatat pemeriksaan persistensi mock dengan 165 record BMKG dan 166 record PVMBG serta 14 respons health layanan lain pada setiap skenario restart. Bukti ini berlaku untuk mock dan volume yang diuji pada waktu tersebut. Bukti tersebut tidak digunakan sebagai hasil restart PostgreSQL kanonis, SQLite webhook, replay feed, atau pemulihan pengiriman notification pada implementasi terbaru.

Prosedur pembuktian persistence diawali dengan mencatat ID serta jumlah data melalui API atau query terkontrol, melakukan restart satu layanan dengan volume tetap, lalu membandingkan data dan health layanan lain. Pengujian restart proses perlu dibedakan dari penghapusan volume dan pemulihan backup. Untuk feed, hasil yang diharapkan justru invalidasi cursor lama sesuai kontrak penyimpanan sementara, bukan kelangsungan seluruh backlog.

Hasil pengujian unit Aggregator pada 9 Oktober 2026 tersedia pada `docs/tex/evidence/aggregator-unit-2026-10-09.jsonl`: tujuh tes tingkat atas pada paket ingestion dan 15 pada paket autentikasi berhasil; paket domain tidak mempunyai berkas tes. Cakupan ingestion

Tabel 5.3: Pemeriksaan penyimpanan yang dapat diulang

Objek	**Observasi yang diperlukan**	**Status bukti**
Hazard dan outbox	Identitas, jumlah record, serta publication pending sebelum dan sesudah restart dengan volume tetap.	Belum diukur ulang.
Subscription webhook	Subscription, log, dan pasangan dedup tetap tersedia setelah restart provider.	Belum diukur ulang.
Replay feed	Cursor lama ditolak setelah restart; event baru memperoleh epoch yang baru.	Kode dan tes tersedia.
Notification	Pembacaan melalui Aggregator dan lifecycle token FCM sesuai kontrak.	Skenario E2E tersedia.

mencakup pemetaan, field tambahan, proyeksi, dan autentikasi HTTP upstream. Hasil ini mendukung kebenaran transformasi data yang diuji, bukan bukti ketahanan storage setelah restart.

Suite `tests/test_e2e_hazards.py`, `tests/test_e2e_webhooks.py`, dan `tests/test_e2e_notifications.py` menyediakan pemeriksaan lintas layanan. Menurut `docs/e2e-testing.md`, suite memerlukan stack aktif dan `BNPB_E2E=1`; tanpa variabel tersebut tes E2E dilewati. Oleh sebab itu, laporan hasil harus membedakan tes yang dieksekusi, dilewati, dan belum diukur. Daftar bukti beserta lingkupnya dirangkum pada Lampiran A.

Dua kriteria tambahan memerlukan demo langsung. Pertama, hentikan satu consumer, build ulang image consumer tersebut, lalu hidupkan kembali tanpa menghentikan stack. Selama proses itu, rekam keberhasilan pembacaan hazard dan penerimaan pada consumer lain, serta bandingkan ID container lain sebelum/sesudah. Kedua, simpan record PVMBG v1 dan v2 pada canonical store, tampilkan keduanya melalui API, dan bandingkan definisi tabel sebelum/sesudah penambahan confidence. Perubahan hanya pada isi JSONB harus dibedakan dari migrasi DDL. Tes mapping E3 mendukung transformasi atribut, tetapi belum menggantikan bukti dua skenario container dan storage ini.

Bab 6

P5: Distribusi Event

6.1 Rancangan Solusi

Aggregator memublikasikan perubahan hazard melalui exchange RabbitMQ bertipe fanout. Setiap consumer memiliki queue sendiri, sehingga satu publication disalin ke Event Live Feed, Webhook Provider, dan Danger Notification. Ketiga layanan dapat memproses publication yang sama untuk tujuan berbeda. Menempatkan ketiganya pada satu queue akan menghasilkan pembagian pekerjaan antar-consumer, bukan penyalinan event ke seluruh layanan.

Tabel 6.1: Topologi distribusi event

Komponen	**Nama contoh/default**	**Peran**
Exchange	`bnpb.hazards.v1`	Fanout durable.
Queue feed	`bnpb.event-live-feed.v1`	Event streaming dan replay.
Queue webhook	`bnpb.webhook-provider.v1`	Callback subscription.
Queue notification	`bnpb.danger-notification.v1`	Push SIAGA/AWAS.
DLQ per consumer	Nama layanan dengan akhiran `.dlq.v1`	Menampung pesan yang ditolak tanpa requeue.

Nama exchange dan queue berasal dari konfigurasi. Producer hanya mendeklarasikan exchange; tiap consumer mendeklarasikan queue durable, dead-letter queue, dan binding miliknya. Routing dead-letter menggunakan default exchange dengan routing key nama DLQ. Ketiga consumer menggunakan prefetch satu pesan dan acknowledgement manual. Pengaturan ini membatasi pesan yang belum diakui, tetapi juga membuat pekerjaan lambat dalam satu consumer menahan pekerjaan berikutnya pada consumer tersebut.

Publication memakai envelope Protobuf `HazardEventPublished`, berisi `event_id`, `schema_version`, sumber, publisher, correlation ID, waktu publication, dan payload hazard kanonis. Implementasi saat ini menggunakan versi schema 1 dan publisher `aggregator`. `event_id` berupa UUID publication dan berbeda dari `hazard_id` yang stabil untuk satu referensi sumber. Pembentukan envelope dapat ditelusuri pada `services/aggregator/internal/ingest/mapper.go`.

Alur penyimpanan dan publication menggunakan pola transactional outbox:

1. Poller memetakan data sumber ke hazard kanonis, lalu membuka transaksi PostgreSQL.
2. Upsert hazard dan penyisipan envelope ke `outbox_events` menggunakan transaksi yang sama, kemudian transaksi di-commit.

3. Publisher mengambil paling banyak 20 publication pending setiap 200 ms dan mengirim pesan persisten ke exchange.
4. Publisher menunggu publisher confirm dari broker sebelum mengisi `published_at`. Kegagalan publish menaikkan retry record.
5. Pada batas retry, contoh konfigurasi lima percobaan, record ditandai gagal dan disalin ke tabel `outbox_dead_letters`.

Implementasi publisher terdapat pada `services/aggregator/internal/broker/publisher.go` dan `services/aggregator/internal/store/outbox_store.go`. Tabel dead-letter outbox merupakan state PostgreSQL producer, berbeda dari DLQ RabbitMQ masing-masing consumer. Kegagalan koneksi awal broker diulang setiap dua detik; retry koneksi tersebut bukan penghitung percobaan publish setiap record.

Setiap consumer menentukan titik acknowledgement sesuai pekerjaannya:

- **Event Live Feed** memvalidasi envelope, menambahkan event ke replay buffer, lalu ack. ID yang masih berada dalam replay window dideduplikasi. Envelope rusak atau tidak didukung langsung di-nack tanpa requeue. Koneksi diulang dengan backoff 1–30 detik.
- **Webhook Provider** mencari subscription yang sesuai filter, memeriksa dedup, membentuk payload sesuai view, dan mengirim HTTP POST dengan tanda tangan HMAC-SHA256 pada `X-BNPB-Signature`. Hasil dicatat ke SQLite dan delivery sukses ditandai untuk dedup. Default dispatcher melakukan lima percobaan dengan timeout 10 detik per request serta jeda 1, 2, 4, dan 8 detik.
- **Danger Notification** memproses envelope, memilih severity SIAGA/AWAS, kemudian mengirim melalui topic FCM. Ack dilakukan setelah pemrosesan berhasil. Parse error Protobuf langsung ditolak; error lain memperoleh satu redelivery sebelum ditolak menuju DLQ.

Kode consumer berada pada `services/event-live-feed/src/adapters/inbound/rabbitmq.rs`, `services/webhook-provider/internal/consumer/consumer.go`, dan `services/danger-notification/BNPB.Notification.Server/Adapters/Inbound/RabbitMqConsumer.cs`. Header trace dibawa melalui AMQP untuk menghubungkan span publish dan consume. Identitas publication membantu menelusuri satu event di beberapa consumer, sementara correlation ID menghubungkannya dengan putaran ingestion.

6.2 Justifikasi dan Alternatif

Fanout dipilih karena semua consumer menerima event hazard yang sama lalu melakukan filter atau proyeksi sesuai fungsi layanan. Producer tidak perlu mengetahui alamat callback maupun perangkat pengguna. Queue terpisah memungkinkan consumer yang berhenti menimbun backlog tanpa memaksa consumer lain berhenti memproses queue masing-masing, selama broker dan resource bersama masih tersedia.

Alternatif panggilan HTTP langsung dari Aggregator ke setiap consumer mengurangi kebutuhan broker, tetapi menjadikan publisher bertanggung jawab atas timeout, retry, dan availability setiap tujuan. Alternatif exchange topic dapat mengurangi lalu lintas dengan routing menurut sumber atau jenis hazard; konsekuensinya ialah penambahan aturan routing dan kesepakatan key. Fanout mempertahankan topologi yang sederhana untuk tiga consumer saat ini.

Alternatif log terdistribusi berbasis offset memberi retensi dan replay yang lebih panjang, tetapi memerlukan pengelolaan partition, offset, dan consumer group. Kebutuhan milestone dapat ditangani RabbitMQ dan outbox PostgreSQL. Keputusan ini tidak menyamakan queue dengan arsip event permanen: event yang telah di-ack tidak lagi menjadi backlog broker, dan replay feed memiliki retensi sendiri.

6.3 Batasan Solusi

Pada transport broker, persistent message, publisher confirm, manual ack, dan redelivery mendukung pola *at-least-once* selama queue sudah terikat, state broker masih tersedia, dan pesan belum dihentikan oleh kebijakan retry atau DLQ. Confirm producer dan ack consumer mengonfirmasi tahap berbeda [7]. Jaminan tersebut tidak sama dengan keberhasilan efek bisnis untuk setiap subscriber. Semantik sistem belum exactly-once. Publisher confirm dan penandaan `published_at` merupakan dua operasi berbeda, sehingga crash di antaranya dapat menyebabkan publication dikirim ulang. Demikian pula, keberhasilan HTTP/FCM dan acknowledgement broker tidak berada dalam transaksi yang sama. Mekanisme dedup harus dinilai per consumer:

1. Feed hanya mendeduplikasi sepanjang replay window dalam memori. Ack setelah append tidak berarti data tersimpan di disk; restart menghapus replay dan himpunan dedup.
2. Webhook menyimpan dedup persisten per pasangan publication dan subscription, tetapi crash setelah penerima menerima POST dan sebelum penandaan dedup masih dapat menghasilkan pengiriman ulang. Penerima juga perlu menangani duplikasi.
3. Notification belum menyimpan dedup publication atau riwayat pengiriman. Redelivery setelah FCM menerima pesan masih dapat menghasilkan push berulang.

Retry webhook memiliki keterbatasan khusus: setelah seluruh percobaan HTTP gagal, consumer mengatur kegagalan pemrosesan dan melakukan nack dengan requeue. Belum ada batas total redelivery pada jalur tersebut. Dengan demikian, lima kegagalan dispatcher tidak otomatis memindahkan event ke DLQ. Komentar kode yang menyebut dispatch failure tidak menghalangi ack tidak sesuai dengan alur kondisi aktual; uraian laporan mengikuti alur yang dieksekusi.

Pola outbox juga belum menutup semua jalur kegagalan aplikasi. Poller mengabaikan error pembentukan envelope dan hasil pemanggilan `EnqueueOutbox`. Transaksi SQL yang sama mendukung konsistensi ketika operasinya berhasil, tetapi belum cukup untuk menjanjikan bahwa setiap hazard pasti mempunyai publication pada seluruh jalur error. Pembaruan watermark dan publikasi berulang perlu diuji secara terpisah.

Publication memakai `mandatory=false`; apabila queue consumer belum pernah dibuat dan diikat ketika pesan dipublikasikan, publisher confirm tidak membuktikan consumer tersebut memperoleh salinannya. Compose belum memasang volume broker secara eksplisit. Karena itu, label durable pada queue dan persistent pada pesan tidak cukup untuk menjanjikan pemulihan setelah container broker dibuat ulang. Pemeriksaan schema juga tidak seragam: feed memvalidasi publisher, ID, sumber, dan payload, sedangkan notification terutama bergantung pada parsing serta akses field yang dipakai.

6.4 Proof-of-Concept dan Keterlacakan

Pada snapshot 9 Oktober 2026 tersedia publisher, tiga consumer, kontrak event, serta skenario pengujian. Belum terdapat hasil pengujian fanout, redelivery, dan pemulihan menyeluruh terbaru yang dilampirkan. Bukti container mock historis tidak digunakan sebagai pengganti bukti RabbitMQ dan ketiga consumer.

Untuk pembuktian yang dapat diulang, ketiga consumer harus dinyalakan lebih dahulu hingga queue dan binding tersedia. Catat satu `event_id` yang dipublikasikan Aggregator, lalu cocokkan penerimaan pada feed, log delivery webhook, dan pemrosesan notification. Gunakan hazard SIAGA/AWAS apabila observasi mencakup pengiriman FCM. Simpan hasil publisher confirm, ack, queue depth, dan trace terkait tanpa menyalin token atau secret.

Skenario berikutnya menghentikan satu consumer setelah binding terbentuk, memublikasikan event, dan memeriksa bahwa consumer lain tetap memproses salinannya. Setelah consumer

Tabel 6.2: Artefak pengujian dan batas cakupannya

Artefak	**Cakupan yang dapat ditelusuri**
`tests/test_e2e_feeds.py`	Penerimaan event SSE, filter seismic/volcanic, dan izin full view.
`tests/test_e2e_webhooks.py`	Lifecycle subscription, tanda tangan ping/event, serta pencatatan delivery.
`tests/test_e2e_notifications.py`	Pembacaan notification dan lifecycle token; tidak dengan sendirinya membuktikan dedup push.
Tes unit adapter consumer	Feed menguji validasi envelope; notification menguji keputusan requeue. Keberadaan tes bukan hasil eksekusi terbaru.

dinyalakan kembali, periksa backlog dan redelivery pada queue miliknya. Uji pesan tidak valid secara terpisah untuk memastikan arah DLQ. Uji replay `event_id` yang sama harus dibedakan dari upsert baru dengan `event_id` berbeda, karena keduanya mempunyai makna deduplikasi yang berbeda.

Eksekusi E2E mengikuti `docs/e2e-testing.md`, dengan stack aktif dan `BNPB_E2E=1`. Hasil yang perlu dilengkapi mencakup penerimaan di tiga consumer, jumlah pengiriman ulang, perilaku DLQ, dan kehilangan replay saat restart feed. Sampai hasil tersebut direkam, status pembuktian P5 adalah mekanisme dan skenario tersedia, sedangkan keberhasilan integrasi serta karakteristik pemulihan terbaru belum diukur.

Untuk menunjukkan penambahan consumer tanpa perubahan producer, buat queue pengamat baru dengan nama unik dan bind ke exchange fanout yang sama melalui RabbitMQ Management atau consumer uji. Setelah binding aktif, tunggu publication baru dan cocokkan `event_id` dengan minimal dua queue layanan yang sudah ada. Rekam konfigurasi binding, payload yang telah disanitasi, dan identitas container Aggregator yang tetap berjalan. Consumer baru menerima publication sejak binding dibuat, bukan otomatis seluruh sejarah sebelum berlangganan. Skenario ini belum dieksekusi sebagai bagian bukti laporan.

Lampiran A

Bukti dan Reproduksi Pengujian

A.1 Inventaris Bukti

Seluruh path pada laporan bersifat relatif terhadap root repository, kecuali path pada kolom berkas berikut yang relatif terhadap `docs/tex/evidence/`. Berkas hasil historis disalin apa adanya dari cache agar tidak hilang ketika cache lokal tidak diikutkan ke Git. Hash SHA-256 pada `evidence/manifest.json` dapat digunakan untuk memeriksa integritas salinan. Laporan ini tidak menyatakan bahwa semua kriteria penerimaan M1 telah lulus.

ID	Waktu	Berkas dan hasil	Cakupan
E1	30 September 2026, 20:13–20:14 WIB	`mock-container-2026-09-30.json`; 10 pemeriksaan lulus.	HTTP ke port Docker kedua mock; termasuk restart. Tidak melewati Aggregator/BNPB API.
E2	6 Oktober 2026, 15:18 WIB	`auth-browser-2026-10-06.json`; 45 pemeriksaan lulus.	Browser dan provider Zitadel nyata pada arsitektur sebelumnya; hasil tidak mewakili integrasi Envoy/API Gateway terbaru.
E3	9 Oktober 2026, 08:33 WIB	`aggregator-unit-2026-10-09.jsonl`; ingest 7 dan auth 15 tes utama lulus.	Unit dan HTTP/JWKS server tes. Paket domain tidak memiliki tes. Subtest tidak dihitung ulang sebagai tes utama.

Dokumen testing lama menyebut 48 pemeriksaan pada versi gateway yang lebih baru, tetapi artefak JSON yang tersedia berisi 45 pemeriksaan pada versi sebelumnya. Laporan menggunakan angka dari artefak yang dapat diperiksa dan tidak menaikkan cakupannya berdasarkan narasi saja. Percobaan tes Auth bridge lokal pada 9 Oktober berhenti sebelum test case berjalan akibat paket `@opentelemetry/api` belum terpasang; status ini dicatat pada manifest sebagai kendala lingkungan.

A.2 Hasil Mock yang Direkam

Pemeriksaan E1	Hasil yang direkam
Health, kontrak payload, korelasi	Lulus; saat diperiksa ada 161 gempa, 41 warning, dan 162 laporan vulkanik. Angka ini mencakup data yang terus dibangkitkan, bukan ukuran seed awal.
Isolasi autentikasi	Tujuh request tidak sah ditolak.
Validasi, correlation ID, filter waktu	Lulus, termasuk batas `since` inklusif dan zona waktu.
Generator dan perubahan skema	Satu gempa baru dan satu laporan v2 baru ditemukan.
Outage dan pemulihan	Satu laporan dibangkitkan selama outage; durasi yang direkam 10,64 detik. Ini bukan uji outage 10–20 menit.
Delay BMKG, 6 sampel	Minimum 104,66 ms; median 105,19 ms; maksimum 119,01 ms.
Delay PVMBG, 6 sampel	Minimum 672,97 ms; median 1642,10 ms; maksimum 2254,86 ms.
Isolasi request paralel	Empat pembacaan PVMBG paralel; health PVMBG 22,29 ms; admin 17,80 ms; data BMKG 113,86 ms.
Persistensi setelah restart	BMKG mempertahankan 165 record dan PVMBG 166; 14 health check service lain berhasil pada masing-masing restart.

Tabel A.2: Ringkasan angka nyata dari E1. Sampel kecil ini bukan benchmark P2.

Cuplikan berikut diambil dari E1; nilainya bukan contoh hasil yang dikarang.


```
"name": "generation_and_schema_evolution",
"status": "passed",
"details": {"new_earthquakes": 1, "new_v2_reports": 1}

"final_state": {
  "health": "both ok",
  "pvmbg_schema": 1,
  "pvmbg_outage": false
}
```


A.3 Hasil Auth dan Aggregator

E2 memuat tiga login hosted UI dengan role berbeda, cookie HttpOnly yang tidak terbaca JavaScript, masa token 60 detik, CSRF, logout, refresh otomatis tanpa login ulang, penolakan token lama, expiry alami, serta persistensi sesi setelah restart bridge. Beberapa nama pemeriksaan memuat frasa *before unfinished backend*; hal ini menegaskan cakupannya masih pengamanan di depan backend yang saat itu belum lengkap. Respons API bisnis yang belum tersedia tidak boleh dihitung sebagai keberhasilan integrasi data.

Contoh baris hasil E2 yang dapat ditelusuri dalam JSON:


```
{"name": "automatic browser refresh without login", "passed": true}
{"name": "naturally expired bearer rejected", "passed": true}
{"name": "SQLite session survives container restart", "passed": true}
```


E3 adalah hasil eksekusi baru pada snapshot kode laporan. Ringkasan tes utama, tanpa menghitung ulang subtest:

Paket	Hasil	Perilaku yang diuji
`ingest`	7 lulus	Fetch BMKG/PVMBG, penolakan autentikasi sumber, severity gempa, warning tsunami, skema v1/v2 dan atribut tambahan, proyeksi Media/full.
`auth`	15 lulus	JWT dengan scope, prioritas claim, expiry, issuer salah, penolakan HS256, rotasi JWKS, token internal, placeholder, mode konfigurasi, provider unavailable dan pemulihan key.
`domain`	Tidak ada tes	Paket berhasil dimuat dalam perintah, tetapi tidak memberikan bukti test case mandiri.

Tabel A.3: Cakupan tes Aggregator aktual pada E3.

A.4 Perintah Reproduksi

Jalankan dari root repository kecuali diberi keterangan lain. Siapkan dependensi dan stack sebagaimana Bab 1. Jangan menganggap hasil *skipped* sebagai lulus.

Tes mock Docker. Setelah kedua mock berjalan, skrip berikut memeriksa HTTP dan restart, lalu memulihkan kontrol PVMBG. Hasil baru menggantikan artefak cache, bukan salinan historis laporan.

```
python tools/test_mock_containers.py --restart-check
```

Tes unit Aggregator. Jalankan dari `services/aggregator`; tidak membutuhkan stack bisnis penuh. Hasil JSON dapat disimpan sebagai bukti baru.

```
go test -json ./internal/ingest ./internal/domain ./internal/auth
```

Tes Auth lokal dan browser. Tes lokal memerlukan dependensi Node workspace yang terpasang. Untuk browser gunakan stack Zitadel yang sesuai dan ikuti prasyarat pada panduan Auth.

```
npx nx run auth-bridge:test
node tools/test_auth_browser.mjs
```

Tes stack penuh. Suite tidak menyalakan Docker secara otomatis. Setelah stack utama siap dan bootstrap menulis konfigurasi service user, aktifkan tes E2E pada PowerShell:

```
$env:BNPB_E2E = "1"
python -m unittest discover -s tests -p "test_e2e_*.py" -v
Remove-Item Env:BNPB_E2E
```

Suite menguji jalur Envoy, token exchange, REST/gRPC, hazard dua sumber, SSE, webhook, serta notifikasi. Test tertentu dapat dilewati ketika data/prasyarat push belum tersedia; jumlah dan alasannya harus dicatat. Uji beban dan isolasi latensi mengikuti `docs/load-testing.md` dengan skrip `tests/load/aggregator.js` dan `tests/load/aggregator-isolation.js`.

A.5 Bukti yang Masih Harus Dilengkapi

Bagian	Pengukuran atau demonstrasi	Artefak yang perlu disimpan
P1	Toggle v1 ke v2 dengan stack tetap hidup dan hazard baru terlihat melalui API BNPB.	Respons sebelum/sesudah, correlation ID, serta identitas/start time container.
P2	Minimal 50 koneksi selama minimal 60 detik; isolasi BMKG saat PVMBG 3 detik; outage dan pulih tanpa restart.	Output k6 aktual, p50/p95/p99, throughput, jumlah request, error tidak terkendali, 429 terpisah, serta log status stale/unavailable.
P3	Tiga role melalui gateway saat ini, Media ditolak saat meminta raw/full, expiry alami, refresh, dan token lama ditolak.	Respons dengan token disembunyikan, waktu expiry/refresh, status HTTP, role, dan hasil browser/E2E terbaru.
P4	Stop/rebuild satu layanan tanpa menjatuhkan jalur independen; record skema lama/baru tetap terbaca tanpa perubahan DDL.	ID container, hasil health/akses sebelum/saat/sesudah, respons data, serta skema tabel.
P5	Event sama diterima minimal dua consumer; satu consumer dihentikan dan dihidupkan; consumer baru ditambahkan tanpa perubahan producer.	Log dengan `event_id` dan correlation ID yang sama, kedalaman queue, ack/redelivery, serta konfigurasi binding consumer tambahan.

Bagian ini adalah daftar kelengkapan bukti, bukan pengganti hasil penerimaan. Setelah pengujian benar-benar dijalankan, tambahkan artefak yang disanitasi ke direktori evidence, perbarui hasil pada bab terkait dan manifest, lalu kompilasi ulang PDF. Kontribusi anggota diisi manual pada `docs/tex/metadata.tex` sebelum pengumpulan.

Daftar Pustaka

[1] Tim Pengampu IF4031. *IF4031 – Tugas Besar – M1*. Dokumen spesifikasi tugas yang diberikan, bagian P1–P5 dan ketentuan laporan.

[2] Kelompok nasgor. *Tugas-Besar-AAT-M1*, snapshot kode commit `64c03cf`, 9 Oktober 2026. Kontrak pada `contracts/`, implementasi pada `services/`, dan orkestrasi pada `compose.yaml`.

[3] OpenID Foundation. *OpenID Connect Core 1.0 incorporating errata set 2*. `https://openid.net/specs/openid-connect-core-1_0.html`.

[4] N. Sakimura, J. Bradley, dan N. Agarwal. *RFC 7636: Proof Key for Code Exchange by OAuth Public Clients*. 2015. `https://www.rfc-editor.org/info/rfc7636/`.

[5] B. Campbell dkk. *RFC 8693: OAuth 2.0 Token Exchange*. 2020. `https://www.rfc-editor.org/info/rfc8693/`.

[6] PostgreSQL Global Development Group. *PostgreSQL Documentation: JSON Types*. `https://www.postgresql.org/docs/current/datatype-json.html`. Referensi konsep JSONB; container implementasi menggunakan PostgreSQL 17.

[7] RabbitMQ. *Consumer Acknowledgements and Publisher Confirms*. `https://www.rabbitmq.com/docs/confirms`.